FIAP - FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA

CURSO SUPERIOR DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Erick Menezes Pereira - RM 570325 | 1TDSPF

Maria Eduarda Escandor - RM 568216 | 1TDSPO

Maria Eduarda Lopes de Lima - RM 572425 | 1TDSPO

Matheus Yudi - RM 571245 | 1TDSPF

Nicolas Sousa da Paixão - RM 574141 | 1TDSPH

ECOUP

API RESTful para registro de ações sustentáveis

DOCUMENTAÇÃO FINAL - SPRINT 4 | JAVA

SÃO PAULO

2026

Erick Menezes Pereira - RM 570325 | 1TDSPF
Maria Eduarda Escandor - RM 568216 | 1TDSPO
Maria Eduarda Lopes de Lima - RM 572425 | 1TDSPO
Matheus Yudi - RM 571245 | 1TDSPF
Nicolas Sousa da Paixão - RM 574141 | 1TDSPH

ECOUP

API RESTful para registro de ações sustentáveis

Trabalho técnico apresentado à FIAP como documentação final da 4ª Sprint da disciplina *Domain Driven Design Using Java*, do curso superior de Tecnologia em Análise e Desenvolvimento de Sistemas. Equipe: ECOUP. Professor: Gilberto Alexandre das Neves.

SÃO PAULO

2026

RESUMO

Este documento apresenta o desenvolvimento e a validação da API RESTful da ECOUP, elaborada para registrar ações sustentáveis e participantes de uma plataforma de engajamento ambiental. A aplicação utiliza Java 21, Spring Boot 4.1.1, JDBC e banco Oracle. A arquitetura está organizada em camadas de transferência de dados (TO), acesso a dados (DAO), regras de negócio (BO) e recursos HTTP (Resource), com filtro de CORS e tratamento de exceções. Foram implementadas operações de cadastro, consulta, atualização e exclusão para usuários e ações. Os novos registros de ações recebem data automática e status REGISTRADA, sem etapa de aprovação. O sistema armazena um endereço de evidência informado pelo participante, mas não realiza upload de vídeo, validação por inteligência artificial ou atribuição real de pontos. A integração com o front-end React foi testada localmente; a API Java está publicada no Render e utiliza um banco Oracle para persistência. Foram registrados testes HTTP de consulta, criação, atualização e exclusão em produção. A documentação reúne a descrição das funcionalidades, modelo entidade-relacionamento, diagrama de classes, tabela de endpoints, evidências visuais e instruções de execução.

Palavras-chave: API RESTful. Java. Spring Boot. Oracle. Sustentabilidade.

SUMÁRIO

1 INTRODUÇÃO 5
2 TECNOLOGIAS E ARQUITETURA 6
3 MODELAGEM DO BANCO DE DADOS 7
4 DIAGRAMA DE CLASSES ATUALIZADO 9
5 SERVIÇOS REST E CONTRATOS HTTP 10
6 VALIDAÇÕES, REGRAS DE NEGÓCIO E TRATAMENTO DE EXCEÇÕES 11
7 EXECUÇÃO LOCAL E PUBLICAÇÃO 12
8 PROTÓTIPO E EVIDÊNCIAS DE FUNCIONAMENTO 13
9 RESULTADOS, LIMITAÇÕES E CONCLUSÃO 17

1 INTRODUÇÃO

A ECOUP é uma proposta de gamificação sustentável elaborada para incentivar atitudes de impacto ambiental positivo. No contexto do desafio acadêmico, o módulo desenvolvido em Java fornece a estrutura de dados e os serviços que permitem registrar pessoas participantes e suas ações. A aplicação integra-se ao protótipo web em React, responsável pelas telas de participação, cadastro de ações, listagem e consulta detalhada.

Esta documentação corresponde à versão efetivamente implementada no projeto de Java, distinguindo as funcionalidades existentes de possibilidades futuras descritas na proposta geral da ECOUP. O sistema atual não realiza análise automática de vídeos nem concede premiações ou pontos calculados no backend.

1.1 Objetivo geral

Desenvolver uma API RESTful com Spring Boot e banco de dados Oracle para executar o ciclo completo de operações CRUD sobre participantes e ações sustentáveis, permitindo sua utilização pelo front-end da ECOUP.

1.2 Objetivos específicos

a) Manter o cadastro de usuários com identificação por código, nome e e-mail.

b) Permitir registrar ações de Mobilidade, Reciclagem ou Energia, relacionadas a um usuário existente.

c) Disponibilizar consultas, atualizações e exclusões por meio de métodos HTTP e respostas apropriadas.

d) Persistir os dados no Oracle e disponibilizar a API em ambiente de nuvem.

1.3 Delimitação do escopo

No escopo concluído, a evidência é representada por um link fornecido no cadastro; não há transferência do arquivo de vídeo para o servidor. Cada nova ação recebe a data corrente e o valor REGISTRADA para o atributo status. A pontuação exibida na interface web tem caráter demonstrativo e não é atribuída ou persistida pelo Java. A consulta de ações e usuários não exige autenticação nesta versão acadêmica, motivo pelo qual a aplicação deve utilizar somente dados fictícios em demonstrações públicas.

2 TECNOLOGIAS E ARQUITETURA

O projeto utiliza Java 21 como versão-alvo da compilação, Spring Boot 4.1.1 para disponibilização dos recursos REST e Oracle Database para persistência por JDBC. O Maven Wrapper permite executar o build a partir do repositório. A aplicação está organizada em pacotes com responsabilidades separadas, acompanhando o padrão desenvolvido nas aulas práticas do MegaFarma.

Quadro 1 - Tecnologias e finalidades

Tecnologia	Finalidade
Java 21	Linguagem e versão configurada no Maven e na imagem Docker.
Spring Boot 4.1.1	Serviço web HTTP, recursos REST e validação de requisições.
Oracle JDBC	Conexão ao banco, consultas e comandos SQL preparados.
Maven Wrapper	Compilação e empacotamento executáveis pelo projeto.
Docker / Render	Empacotamento e publicação da API Java na nuvem.
React (projeto separado)	Cliente web que consome a API REST.

Fonte: elaboração própria (2026).

2.1 Organização em camadas

O pacote to representa os dados transportados, o pacote dao concentra a persistência e a conexão JDBC, o pacote bo implementa verificações de negócio e o pacote resource mapeia os endpoints HTTP. A classe EcoupApplication inicializa o Spring Boot; CorsFilter permite requisições de clientes web por métodos HTTP configurados.

Figura 1 - Fluxo da arquitetura da solução



O front-end envia requisições HTTP; o DAO utiliza JDBC e SQL no Oracle.

Fonte: elaboração própria (2026).

2.2 Estrutura de diretórios

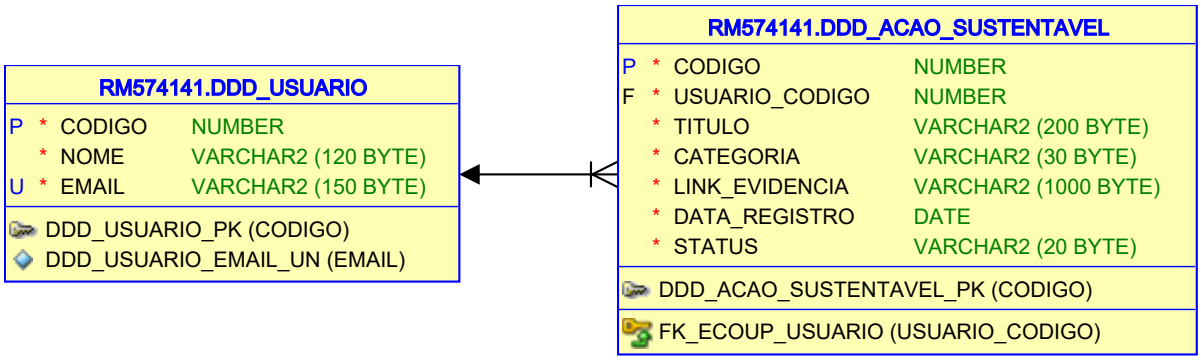
Pacotes: to (modelos); dao (persistência e ConnectionFactory); bo (regras); resource (REST e CorsFilter). A classe EcoupApplication inicializa o serviço.

O arquivo application.properties identifica a aplicação e utiliza a porta definida pelo ambiente por meio da expressão ${PORT:8080}. O Dockerfile compila o JAR e executa a aplicação com Java 21.

3 MODELAGEM DO BANCO DE DADOS

O modelo lógico empregado pela ECOUP contém duas tabelas relacionadas: DDD_USUARIO, que guarda os participantes, e DDD_ACAO_SUSTENTAVEL, que registra ações associadas aos participantes. A chave estrangeira USUARIO_CODIGO referencia DDD_USUARIO.CODIGO. A relação é de um usuário para muitas ações (1:N).

Figura 2 - Modelo de entidade-relacionamento (MER) da ECOUP



Uma pessoa usuária pode registrar diversas ações;
cada ação referencia exatamente um usuário pelo campo USUARIO_CODIGO.

Fonte: modelo exportado do Oracle SQL Developer Data Modeler (2026).

3.1 Dicionário de dados

Quadro 2 - Atributos da tabela DDD_USUARIO

Atributo	Tipo	Função
CODIGO	NUMBER	Identificador (chave primária).
NOME	VARCHAR2(120)	Nome do participante.
EMAIL	VARCHAR2(150)	E-mail; espera-se unicidade no banco.

Quadro 3 - Atributos da tabela DDD_ACAO_SUSTENTAVEL

Atributo	Tipo	Função
CODIGO	NUMBER	Identificador (chave primária).
USUARIO_CODIGO	NUMBER	Chave estrangeira para usuário.
TITULO	VARCHAR2(200)	Descrição resumida da atitude.
CATEGORIA	VARCHAR2(30)	Mobilidade, Reciclagem ou Energia.
LINK_EVIDENCIA	VARCHAR2(1000)	URL informada pelo participante.
DATA_REGISTRO	DATE	Data definida no cadastro.
STATUS	VARCHAR2(20)	REGISTRADA para novos cadastros.

Fonte: elaboração própria, conforme o código e a estrutura documentada do projeto (2026).

3.2 Integridade e persistência

As operações DAO utilizam PreparedStatement com parâmetros posicionais para executar comandos SQL e mapear os resultados para os objetos TO. O retorno de chaves geradas é utilizado no cadastro, permitindo que o React consulte os detalhes pelo identificador fornecido pela API. Cada operação utiliza o fechamento automático de conexões e recursos JDBC com try-with-resources.

A integridade referencial protege o vínculo entre usuários e ações: a exclusão de um usuário que ainda possua ações associadas pode ser recusada pelo Oracle, situação tratada como HTTP 409 pela API. Da mesma forma, a tentativa de cadastrar e-mail repetido é tratada como conflito quando a restrição de unicidade está presente. Não existe exclusão automática em cascata implementada no Java.

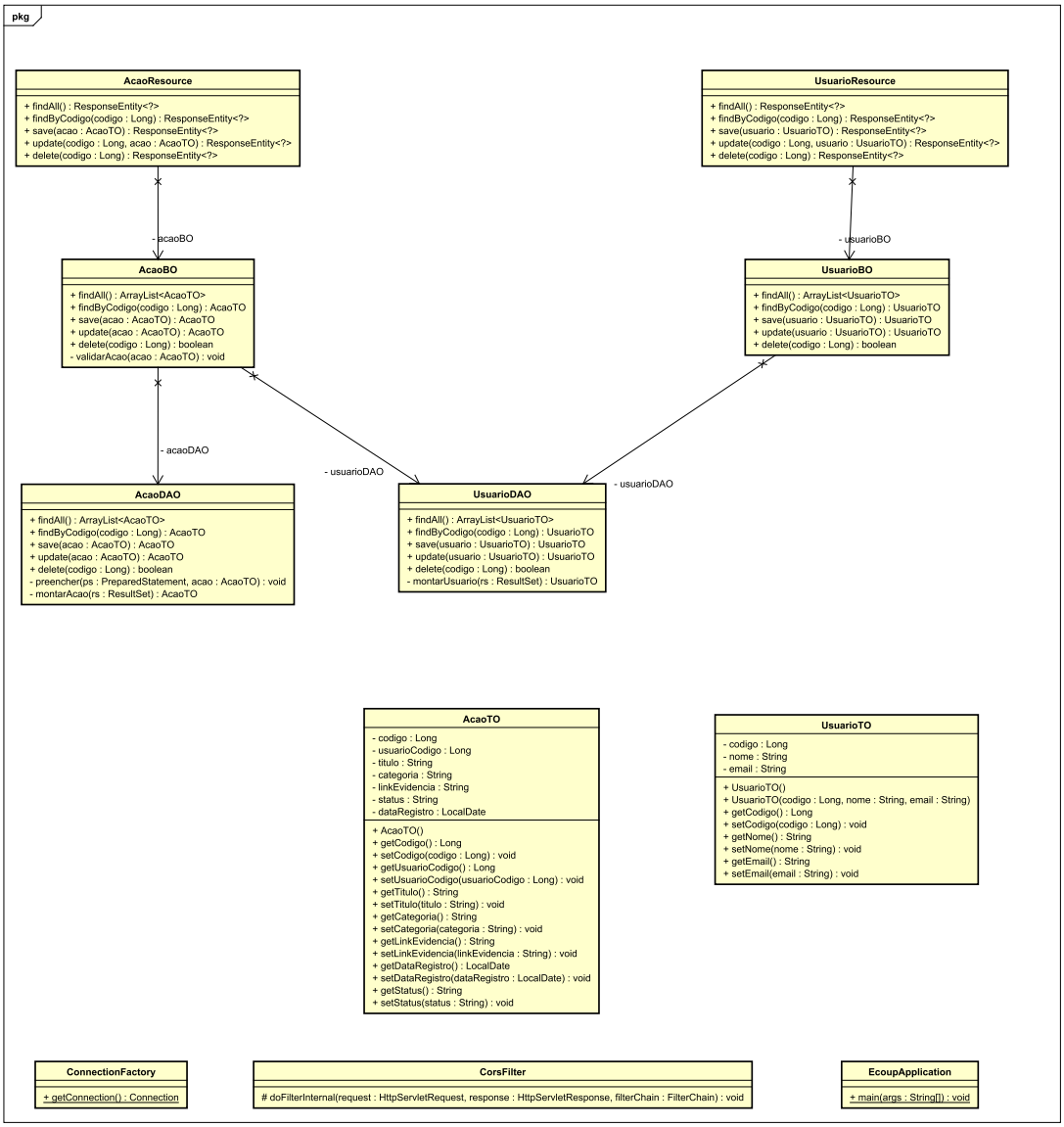
3.3 Observação sobre registros anteriores

A implementação atual grava novos cadastros com status REGISTRADA. As ações antigas, criadas em versões anteriores com status PENDENTE, permanecem no banco com o valor originalmente persistido. Não existe rotina de migração automática para alterar esses dados históricos.

4 DIAGRAMA DE CLASSES ATUALIZADO

A Figura 3 representa as classes efetivamente presentes no pacote br.com.fiap do projeto. O diagrama segue a representação UML com classes, atributos, operações e relações de dependência. Os Resources dependem das classes BO; estas utilizam os DAOs; os DAOs manipulam objetos TO e obtêm conexões pela ConnectionFactory. AcaoBO também consulta UsuarioDAO para verificar a existência do participante.

Figura 3 - Diagrama UML de classes - API ECOUP



Fonte: diagrama de classes elaborado no Astah a partir do Java da ECOUP (2026).

5 SERVIÇOS REST E CONTRATOS HTTP

Os recursos foram organizados sob as rotas /api/usuarios e /api/acoes. GET realiza consultas; POST cria novos registros; PUT atualiza dados existentes; DELETE remove registros. O servidor utiliza formatos JSON para as operações que possuem corpo de resposta. As tabelas a seguir apresentam os códigos de sucesso e os principais códigos de erro previstos pelo código.

Quadro 4 - Endpoints de usuários

Método	URI	Descrição	HTTP esperado
GET	/api/usuarios	Listar usuários	200; 500
GET	/api/usuarios/{codigo}	Buscar por código	200; 404; 500
POST	/api/usuarios	Cadastrar usuário	201; 400; 409; 500
PUT	/api/usuarios/{codigo}	Atualizar usuário	200; 400; 404; 409; 500
DELETE	/api/usuarios/{codigo}	Excluir usuário	204; 404; 409; 500

Fonte: recursos de UsuarioResource.java; 400 pode ser retornado por @Valid (2026).

Quadro 5 - Endpoints de ações sustentáveis

Método	URI	Descrição	HTTP esperado
GET	/api/acoes	Listar ações	200; 500
GET	/api/acoes/{codigo}	Buscar por código	200; 404; 500
POST	/api/acoes	Cadastrar ação	201; 400; 500
PUT	/api/acoes/{codigo}	Atualizar ação	200; 400; 404; 500
DELETE	/api/acoes/{codigo}	Excluir ação	204; 404; 500

Fonte: recursos de AcaoResource.java; respostas conforme o código (2026).

5.1 Exemplos de requisição e resposta

Exemplo de POST de usuário:

```
{ "nome": "Participante Teste", "email": "teste.ecoup@example.com" }
```

Exemplo de POST de ação:

```
{ "usuarioCodigo": 1, "titulo": "Fui de bicicleta", "categoria": "Mobilidade",
"linkEvidencia": "https://exemplo.com/video" }
```

Na criação de ações, codigo, dataRegistro e status são preenchidos pelo servidor e devolvidos na resposta. O registro é considerado realizado imediatamente, sem fluxo de revisão ou aprovação.

6 VALIDAÇÕES, REGRAS DE NEGÓCIO E TRATAMENTO DE EXCEÇÕES

6.1 Validações aplicadas

As classes TO utilizam anotações de Bean Validation para impedir entradas incompatíveis com campos obrigatórios. UsuarioTO define nome obrigatório de até 120 caracteres e e-mail obrigatório, com formato de e-mail e até 150 caracteres. AcaoTO exige usuarioCodigo positivo, título de até 200 caracteres, categoria e link de evidência obrigatórios, com limite de 1000 caracteres para o link.

AcaoBO verifica se o participante informado existe e se a categoria pertence ao conjunto Mobilidade, Reciclagem ou Energia. Novas ações são preenchidas com LocalDate.now() e status REGISTRADA. A URL fornecida é armazenada como texto, sem processamento do conteúdo do vídeo.

6.2 Tratamento de erros HTTP

Quadro 6 - Respostas de erro e situações relevantes

Situação	Código	Comportamento
Campos inválidos no corpo	400	Bean Validation rejeita a requisição.
Categoria inválida ou usuário da ação inexistente	400	Mensagem simples da regra de negócio.
Consulta a registro inexistente	404	Mensagem indicando registro não encontrado.
E-mail repetido (restrição UNIQUE)	409	Conflito no cadastro/atualização.
Excluir usuário com ações (restrição FK)	409	Conflito por vínculo referencial.
Falha imprevista no Oracle	500	Mensagem genérica, sem detalhes internos do banco.

Fonte: implementação das classes BO, DAO e Resource (2026).

Os Resources utilizam try/catch, ResponseEntity e HttpStatus, seguindo os mecanismos praticados em sala. Os DAOs identificam os códigos Oracle ORA-00001 e ORA-02292 no recurso de usuários. A ocorrência real dos conflitos depende das restrições existentes no esquema Oracle.

6.3 Observação de segurança

A API possui endpoints públicos e não implementa controle de acesso ou autenticação nesta versão. Os testes apresentados empregam dados fictícios; a utilização com dados reais exigiria proteção de acesso, política de privacidade e outras salvaguardas não incluídas no escopo acadêmico.

7 EXECUÇÃO LOCAL E PUBLICAÇÃO

7.1 Ambiente e ferramentas

Ferramentas utilizadas: IntelliJ IDEA ou outro ambiente com suporte a Maven, JDK 21, PowerShell/CMD, Oracle SQL Developer, Insomnia, Git/GitHub e Render. O projeto foi executado também localmente com uma instalação de Java 25, embora sua configuração oficial de compilação e sua imagem Docker utilizem Java 21.

7.2 Passo a passo para executar localmente

1. Abrir o repositório no IntelliJ (ou outra IDE) pela pasta que contém o pom.xml. 2. Conferir as duas tabelas no SQL Developer. 3. Definir as variáveis DB_URL, DB_USER e DB_PASSWORD. 4. Compilar com Maven e executar o JAR. 5. Consultar /api/usuarios ou /api/acoes com navegador ou Insomnia.

Quadro 7 - Comandos de execução em PowerShell

Etapa	Comando
Variável Oracle	$env:DB_URL="jdbc:oracle:thin:@SERVIDOR:1521:SID"
Usuário	$env:DB_USER="USUARIO"
Senha	$env:DB_PASSWORD="SENHA"
Compilação	.\mvnw.cmd clean -DskipTests package
Inicialização	java -jar target/ecoup-0.0.1-SNAPSHOT.jar

Fonte: procedimentos usados no desenvolvimento, com credenciais substituídas por exemplos (2026).

7.3 Deploy no Render

O Dockerfile utiliza eclipse-temurin:21-jdk-alpine, compila o JAR e inicia o Spring Boot. No Render, o serviço Docker recebe DB_URL, DB_USER e DB_PASSWORD como variáveis de ambiente. O parâmetro server.port=${PORT:8080} utiliza a porta da hospedagem.

URL pública da API: https://ecoup-sprint4-java.onrender.com

Exemplo de consulta: https://ecoup-sprint4-java.onrender.com/api/acoes

A rota raiz / não possui recurso de página inicial e pode apresentar HTTP 404, sem impedir o funcionamento das rotas /api. O serviço foi registrado como Live no Render, e consultas a dados Oracle foram demonstradas por meio de capturas de tela da execução remota.

8 PROTÓTIPO E EVIDÊNCIAS DE FUNCIONAMENTO

As imagens desta seção são recortes de capturas reais fornecidas durante os testes em 3 de novembro de 2026. Foram selecionadas apenas as evidências essenciais para verificar a interface e o comportamento da API, evitando reproduzir janelas completas do navegador e do Insomnia. Não houve alteração das respostas HTTP exibidas.

Figura 4 - Detalhes de uma ação na interface ECOUP

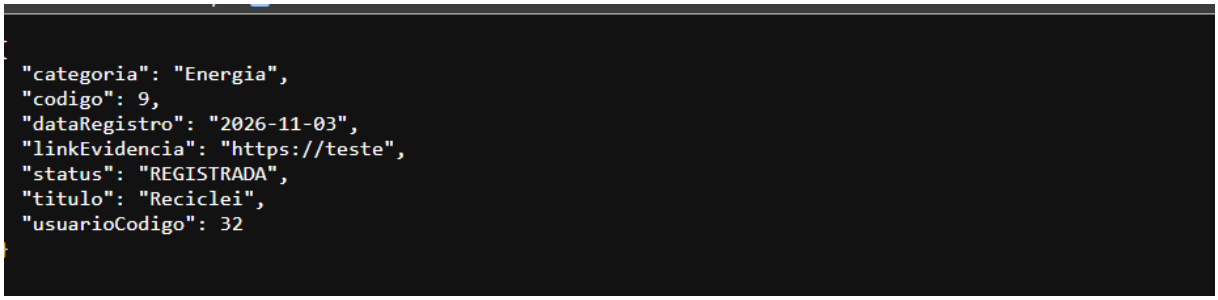


Fonte: captura de tela do protótipo ECOUP (2026).

O front-end React exibe os detalhes da ação registrada. A pontuação apresentada na interface é ilustrativa; não é gerada nem persistida pela API Java.

8.1 Consulta de ação na API pública

Figura 5 - Consulta GET /api/acoes/9 no Render

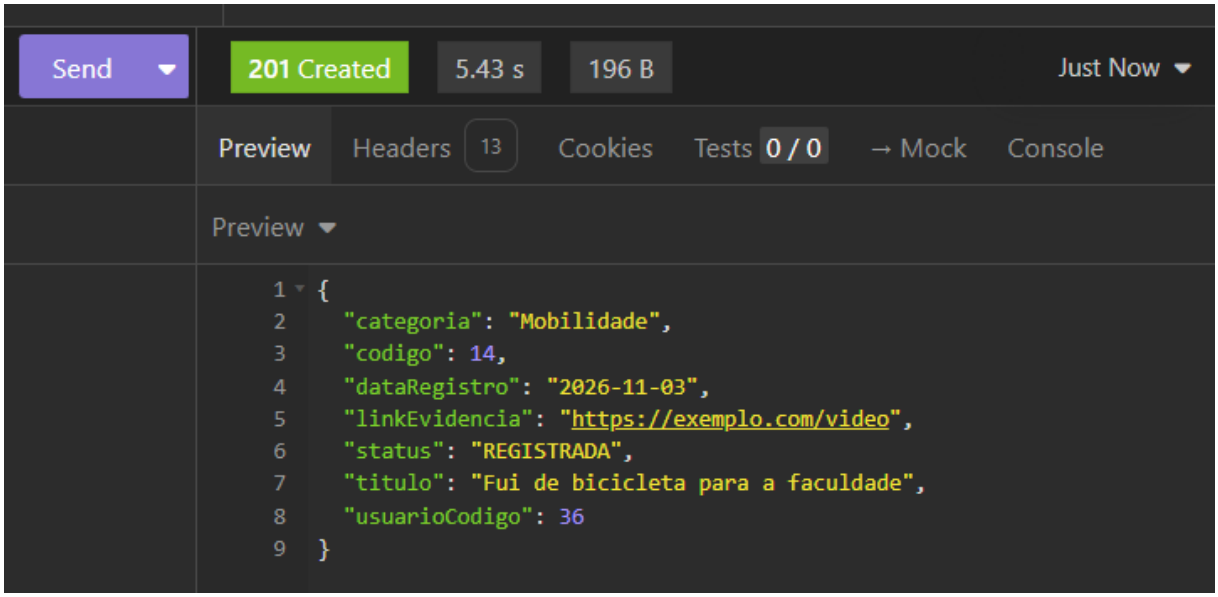


Fonte: recorte da captura de teste em navegador (2026).

O endpoint consulta a ação por código e apresenta os campos persistidos pelo Oracle, incluindo categoria, título, data e status REGISTRADA. A rota /api/acoes também disponibiliza a listagem completa.

8.2 Cadastro de ação pelo Insomnia

Figura 6 - Resposta 201 Created de POST /api/acoes

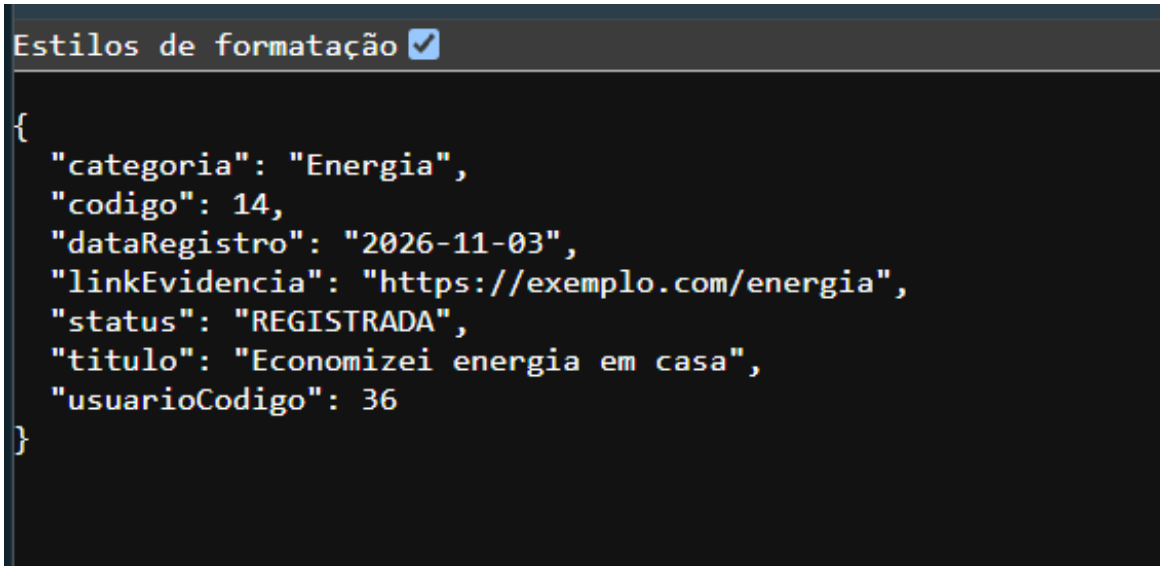


Fonte: recorte da captura original no Insomnia (2026).

A criação do recurso retorna HTTP 201 Created, o código da nova ação, o identificador do usuário e o status REGISTRADA. A requisição completa foi executada contra a API publicada no Render.

8.3 Atualização e exclusão

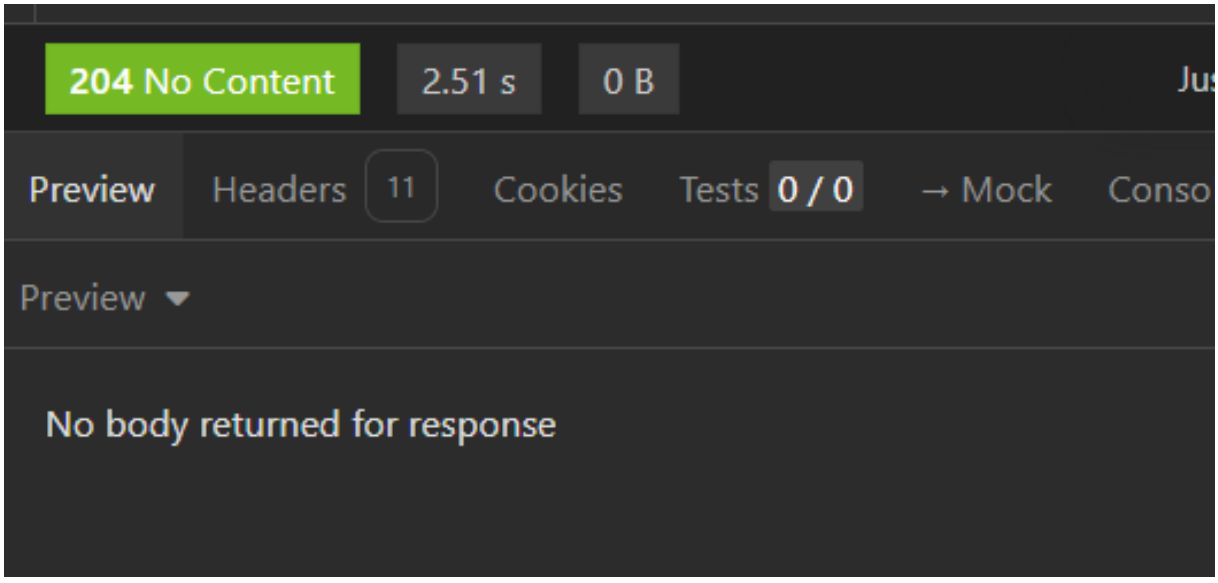
Figura 7 - GET após atualização da ação por PUT



Fonte: captura de teste com nova categoria Energia (2026).

Após a atualização PUT, nova consulta GET confirmou o título e a categoria modificados.

Figura 8 - Resposta 204 No Content em DELETE /api/acoes/14



Fonte: recorte da captura de teste no Insomnia (2026).

A ação fictícia foi removida com HTTP 204 No Content; em seguida, o participante fictício associado também foi excluído, respeitando a relação por chave estrangeira.

9 RESULTADOS, LIMITAÇÕES E CONCLUSÃO

9.1 Síntese das verificações

Quadro 8 - Conferência das funcionalidades da Sprint 4 Java

Exigência	Implementação/ evidência
API RESTful com Spring Boot	Endpoints GET, POST, PUT e DELETE para usuários e ações.
Camadas Model, DAO, BO e Resource	Onze classes organizadas em pacotes e documentação UML.
Oracle e MER	Duas tabelas relacionadas por chave estrangeira.
Validações e erros	Bean Validation, verificações na BO e respostas 400/404/409/500.
Integração com front-end	React consulta e cadastra recursos via API em ambiente local.
Deploy na nuvem	Serviço Docker no Render com consultas e gravações demonstradas.
Documentação	PDF com endpoints, MER, UML, telas e procedimentos.

Fonte: elaboração própria com base na implementação e nas evidências de testes (2026).

9.2 Limitações conhecidas

O módulo desenvolvido não autentica usuários, não processa o conteúdo de vídeos, não calcula pontuação ou impacto ambiental, não aplica reconhecimento automático de ações e não gerencia benefícios financeiros. O link de evidência é apenas armazenado. O histórico pode conter ações antigas com status PENDENTE, pois a alteração para REGISTRADA vale para novos registros.

9.3 Conclusão

A implementação Java da ECOUP atende ao escopo funcional definido para o protótipo da Sprint 4: disponibiliza uma API RESTful estruturada, com CRUD completo, persistência no Oracle, validações essenciais, tratamento de exceções e integração com a interface em React. Os testes registrados comprovam consultas, inclusões, alterações e exclusões utilizando a URL pública do Render. O projeto preserva os padrões didáticos empregados nas aulas práticas do MegaFarma, adaptados ao domínio de participantes e ações sustentáveis.